ML Practicals 1–6: Line-by-Line Explanation

Practical 1: PCA for Dimensionality Reduction (Wine dataset)

1. Aim and idea

The Wine dataset has 178 samples and 13 chemical features (Alcohol, Malic_Acid, Ash, ... Proline) plus a label `Customer_Segment` (3 classes: 59, 71, 48 samples). **PCA (Principal Component Analysis)** creates new, uncorrelated axes (principal components) that are linear combinations of the original features, ordered by how much variance they capture. We keep only the first few components and so reduce 13 dimensions to fewer with little information loss.

2. Code line by line

Cell 1: imports

- `import pandas as pd` : tables (DataFrames) and CSV reading.
- `import numpy as np` : numerical arrays, `cumsum`, `argmax`, `arange`.
- `import matplotlib.pyplot as plt` : plotting.
- `from sklearn.preprocessing import StandardScaler` : scales each feature to mean 0 and standard deviation 1.
- `from sklearn.decomposition import PCA` : the PCA algorithm.
- `plt.rcParams["figure.figsize"] = (10, 6)` : sets the default plot size to 10 x 6 inches.

Cell 2: load data

- `FILE = "Wine.csv"` : file name stored in a variable.
- `df = pd.read_csv(FILE)` : reads the CSV into a DataFrame.
- `print("Dataset shape:", df.shape)` : prints (178, 14), i.e. 178 rows, 14 columns (13 features + target).
- `display(df.head())` : shows the first 5 rows.

Cell 3: exploring the data

- `df.columns.tolist()` : list of column names.
- `df.dtypes.to_frame("dtype")` : data type of each column (all numeric).
- `df.isna().sum().to_frame("missing")` : number of missing values per column (all 0, so no cleaning needed).
- `df.describe().T` : count, mean, std, min, quartiles, max of every column, transposed for easy reading.
- `df["Customer_Segment"].value_counts().sort_index()` : class balance (1: 59, 2: 71, 3: 48).

Cell 4: separate features and target

- `target_col = "Customer_Segment"` : name of the label column.
- `X = df.drop(columns=[target_col])` : all 13 feature columns.

- `y = df[target_col]` : labels. PCA does **not** use y; it is kept only to colour the final plot.

Cell 5: standardisation

- `scaler = StandardScaler()` : creates the scaler.
- `X_scaled = scaler.fit_transform(X)` : learns the mean and std of each column and applies z = (x - mean) / std. Needed because PCA maximises variance, and features with large units (Proline is in the hundreds, Hue is about 1) would otherwise dominate.
- `X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)` : puts the NumPy array back into a labelled table.

Cell 6: full PCA and variance table

- `pca_full = PCA()` : with no `n_components`, keeps all 13 components.
- `X_pca_full = pca_full.fit_transform(X_scaled)` : finds the components and projects the data onto them.
- `explained_variance_ratio = pca_full.explained_variance_ratio_` : fraction of total variance carried by each component (PC1 = 0.362, PC2 = 0.192, PC3 = 0.111 ...).
- `cumulative_variance = np.cumsum(...)` : running total (PC1+PC2 = 0.554, first 8 = 0.920, all 13 = 1.0).
- `variance_table = pd.DataFrame({...})` : builds a table with Component number, ratio and cumulative variance.

Cell 7: the two line plots

- `plt.plot(Component, Explained_Variance_Ratio, marker="o")` : scree plot.
- `plt.axhline(0.90, linestyle="--")` and `plt.axhline(0.95, ...)` : horizontal dashed guide lines at 90% and 95%.
- `plt.title / xlabel / ylabel / xticks / legend / tight_layout / show` : labels, one tick per component, legend, layout fix, then display.

Cell 8: choose number of components

- `np.argmax(cumulative_variance >= 0.90) + 1` : the comparison gives True/False values; `argmax` returns the index of the first True; `+1` converts index to a count. Result: **8 components** for 90%.
- Same with 0.95 gives **10 components** for 95%.

Cell 9: PCA with 2 components

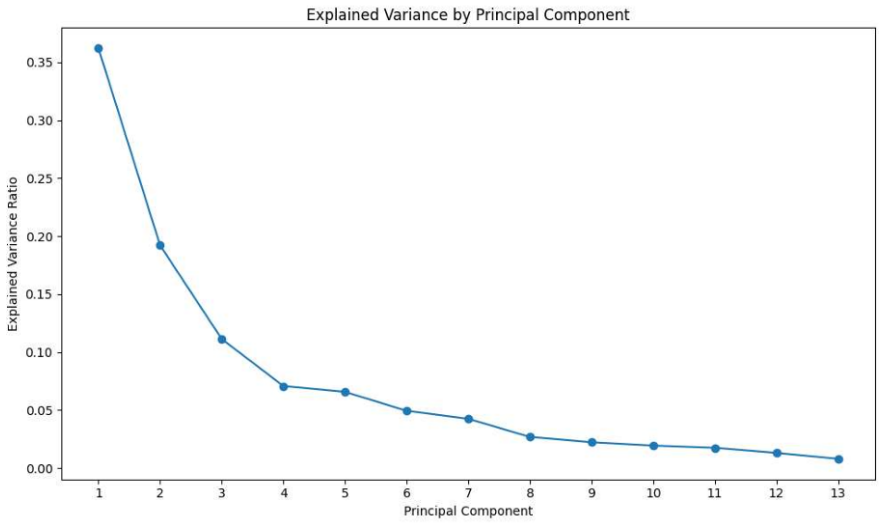
- `pca_2 = PCA(n_components=2)` : keep only PC1 and PC2.
- `X_pca_2 = pca_2.fit_transform(X_scaled)` : the 178 x 2 reduced data.
- `pca_df = pd.DataFrame(X_pca_2, columns=["PC1","PC2"])` and `pca_df["Customer_Segment"] = y.values` : attach the class label for colouring.
- The three `print` lines show PC1 = 0.3620, PC2 = 0.1921, total = **0.5541** (55.4% of the information in 2 axes).

Cell 10: scatter plot

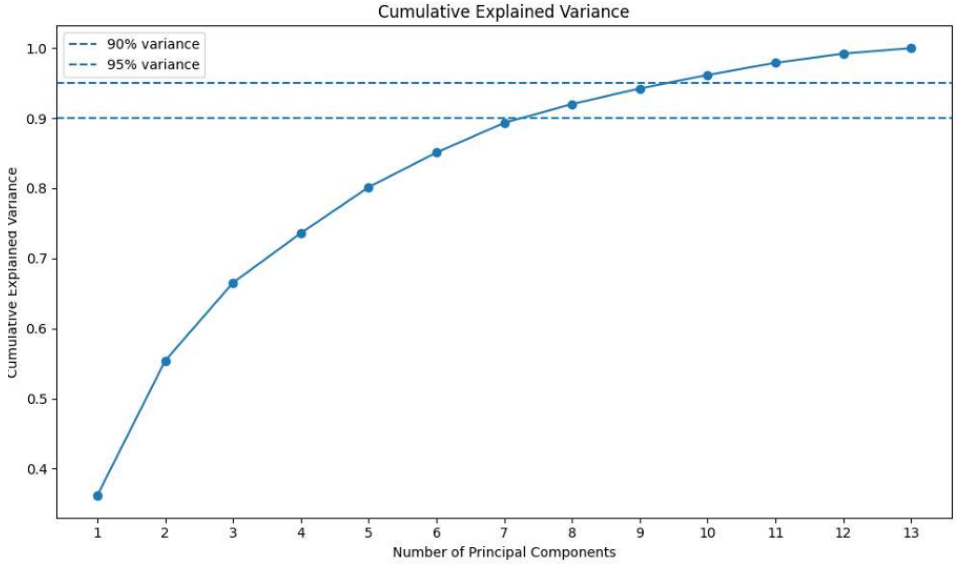
- `for segment in sorted(pca_df["Customer_Segment"].unique())` : loop over classes 1, 2, 3.
- `subset = pca_df[pca_df["Customer_Segment"] == segment]` : rows of that class.
- `plt.scatter(subset["PC1"], subset["PC2"], label=..., alpha=0.75)` : draw that class as semi-transparent points in its own colour.
- `plt.grid(True)`, `plt.legend()` : grid and legend.

3. What the diagrams represent

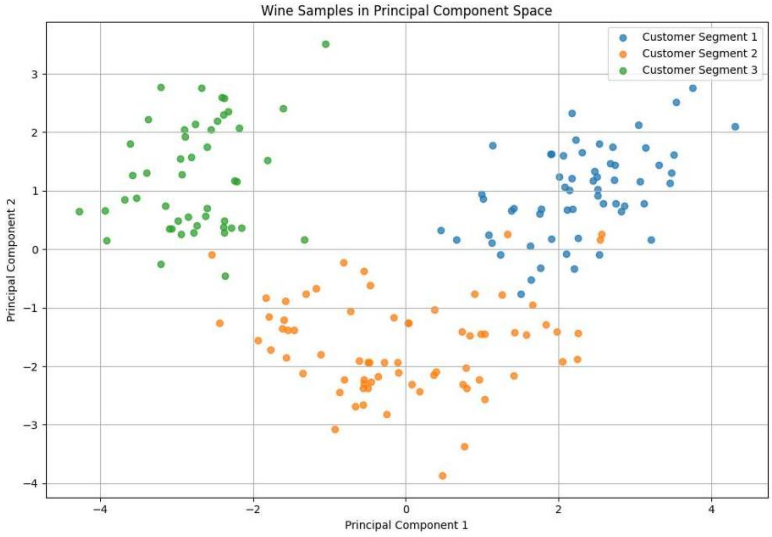
1. **Scree plot (Explained Variance by Principal Component):** a falling curve. PC1 carries 36%, PC2 19%, and the bars after PC4 are small. The "bend" shows that only the first few components matter.



2. **Cumulative explained variance:** a rising curve that reaches the 90% dashed line at 8 components and the 95% line at 10. It tells you how many components to keep for a chosen information level.



3. **PC1 vs PC2 scatter plot:** each dot is one wine sample in the new 2-D space. The three colours form three fairly separate groups: Segment 3 (green) on the left, Segment 1 (blue) on the upper right, Segment 2 (orange) at the bottom/middle. This shows PCA keeps class structure even though it never saw the labels. Slight overlap between Segment 1 and 2 exists because 2 components hold only 55% of the variance.



4. Top 5 viva questions

Q1. What is PCA and why is it used? PCA is an unsupervised linear technique that transforms correlated features into a smaller set of uncorrelated principal components that retain maximum variance. It is used for dimensionality reduction, noise removal, visualisation and speeding up models.

Q2. Why did you standardise the data before PCA? PCA picks directions of maximum variance, so features with a larger numeric scale (like Proline) would dominate. StandardScaler gives every feature mean 0 and std 1 so each contributes fairly.

Q3. What is explained variance ratio and how did you choose the number of components? It is the share of total variance carried by each component (eigenvalue / sum of eigenvalues). I took the cumulative sum and kept the smallest number of components reaching 90% variance, which was 8 (10 for 95%). The scree plot elbow can also guide the choice.

Q4. Explain the steps of PCA mathematically. (1) Standardise the data. (2) Compute the covariance matrix. (3) Find its eigenvalues and eigenvectors. (4) Sort eigenvectors by decreasing eigenvalue; these are the principal components. (5) Project the data onto the top k eigenvectors. (sklearn actually uses SVD, which gives the same result.)

Q5. What are the limitations of PCA, and how is it different from feature selection? PCA captures only linear relationships, is sensitive to scale and outliers, and the new components are hard to interpret. Feature selection keeps a subset of the original features, whereas PCA builds new features from combinations of all original ones.

Practical 2: Statistical and Regression Analysis (Pima Diabetes data)

1. Aim and idea

The Pima diabetes data has 768 patients and 9 columns (Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age, Outcome). Outcome is binary (500 non-diabetic, 268 diabetic). The practical covers: univariate statistics, bivariate linear regression (Glucose to BMI), bivariate logistic regression (Glucose to Outcome), multiple linear regression (many features to BMI) and multiple logistic regression (many features to Outcome).

2. Code line by line

Cell 2: imports

- `skew, kurtosis` from `scipy.stats` : imported for shape statistics (the notebook later uses pandas `.skew()` and `.kurt()` instead).
- `train_test_split` : splits data into train and test parts.
- `LinearRegression, LogisticRegression` : the two models.
- `mean_squared_error, mean_absolute_error, r2_score` : regression metrics.
- `accuracy_score, precision_score, recall_score, f1_score` : classification metrics.
- `confusion_matrix, ConfusionMatrixDisplay` : confusion matrix and its plot.

Cell 3: load

- `pima = pd.read_csv("diabetes.csv")` reads the file; `pima.shape` is (768, 9); `pima.head()` shows 5 rows. (Zeros in columns such as Insulin were already replaced by median-type values, e.g. 169.5 and 102.5.)

Cell 4: overview

- Lists columns, dtypes, missing values (none), `describe().T` summary and `value_counts()` of Outcome (0: 500, 1: 268, so the data is imbalanced).

Cell 5: frequency table of the categorical variable

- `categorical_cols = ["Outcome"]` then a loop printing `pima[col].value_counts().sort_index()`, which counts how often each category occurs.

Cell 6: frequency tables of numeric variables

- `selected_for_frequency = ["Pregnancies","Glucose","BMI","Age"]`; for each, `value_counts().sort_index().head(20)` shows the first 20 distinct values with their counts (e.g. 111 women with 0 pregnancies, 63 patients aged 21).

Cell 7: univariate statistics

- `numeric_cols = pima.select_dtypes(include=np.number).columns` : all numeric columns.
- A DataFrame is built with `.mean()` (average), `.median()` (middle value), `.mode().iloc[0]` (most frequent value), `.var()` (spread squared), `.std()` (spread), `.skew()` (asymmetry) and `.kurt()` (tail heaviness / peakedness).

- `display(stats.round(4))` prints it to 4 decimals. Example: Insulin has a large variance (7938.96); Age has mean 33.24 but median 29 and mode 22, so it is right-skewed.

Cell 8: histograms

- `for col in ["Glucose","BMI","Age","BloodPressure"]` : one figure per variable.
- `plt.hist(pima[col].dropna(), bins=20, edgecolor="black")` : splits the range into 20 bins and draws bar heights as frequency.

Cell 9: bivariate linear regression (Glucose to BMI)

- `X_linear = pima[["Glucose"]]` : predictor (double brackets keep it 2-D, as sklearn requires).
- `y_linear = pima["BMI"]` : continuous response.
- `linear_model = LinearRegression()` then `.fit(X_linear, y_linear)` : finds the best straight line by least squares.
- `linear_pred = linear_model.predict(X_linear)` : fitted values.
- Prints the equation **BMI = 25.9443 + 0.0533 x Glucose**, **R^2 = 0.0558**, MAE = 5.2844, RMSE = 6.6815. The low R^2 means Glucose explains only about 5.6% of BMI variation, a very weak relationship.

Cell 10: regression plot

- `plt.scatter(Glucose, BMI, alpha=0.5)` : the data points.
- `order = np.argsort(pima["Glucose"].values)` : sort indices by Glucose so the line is drawn left to right without zig-zagging.
- `plt.plot(Glucose[order], linear_pred[order], linewidth=2)` : the fitted line.

Cell 11: bivariate logistic regression (Glucose to Outcome)

- `log_model = LogisticRegression(max_iter=1000)` : classifier; `max_iter` allows enough optimisation steps to converge.
- `log_prob = log_model.predict_proba(X_log)[:, 1]` : probability of class 1 (diabetic).
- `log_pred = log_model.predict(X_log)` : class labels (probability >= 0.5 gives 1).
- Prints coefficient 0.0409, intercept -5.7488, accuracy **0.7448**, precision 0.6875, recall **0.4925**, F1 0.5739. The model is correct about 74% of the time, but finds fewer than half of the real diabetics.

Cell 12: logistic plots

- Scatter of the 0/1 outcomes (alpha=0.25 so overlapping points are visible), the S-shaped predicted-probability curve, and `plt.axhline(0.5, linestyle="--")` as the decision threshold.
- `ConfusionMatrixDisplay.from_predictions(y_log, log_pred)` draws the confusion matrix.

Cell 13: multiple linear regression (7 features to BMI)

- `multiple_features = [...]` : 7 predictors (all except BMI and Outcome).
- `multi_model.fit(X_multi, y_multi)` then predict; a coefficient table is displayed with `multi_model.coef_`. Intercept 11.7834, **R^2 = 0.3892**, MAE 4.1826, RMSE 5.3738. Using many

variables raised R^2 from 0.056 to 0.389. DiabetesPedigreeFunction (1.71) and SkinThickness (0.39) have the largest coefficients (note the features are on different scales, so coefficient size alone is not importance).

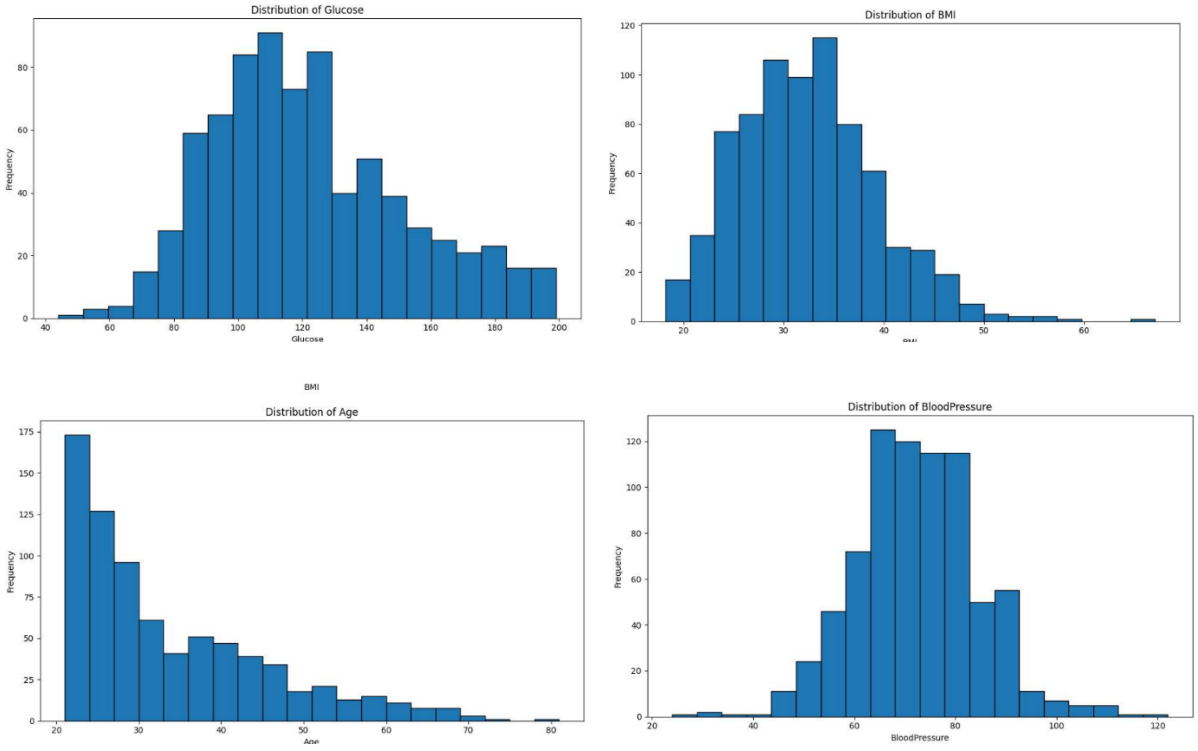
Cell 14: multiple logistic regression

- `train_test_split(X_ml, y_ml, test_size=0.20, random_state=42, stratify=y_ml)` : 80% train, 20% test; `random_state` makes the split repeatable; `stratify` keeps the 500:268 class ratio in both parts.
- Fit on train, predict on test. Accuracy **0.7013**, precision 0.5833, recall 0.5185, F1 0.5490. Coefficients: DiabetesPedigreeFunction 0.56 and Pregnancies 0.09 are the largest.
- Confusion matrix of the test set is plotted.

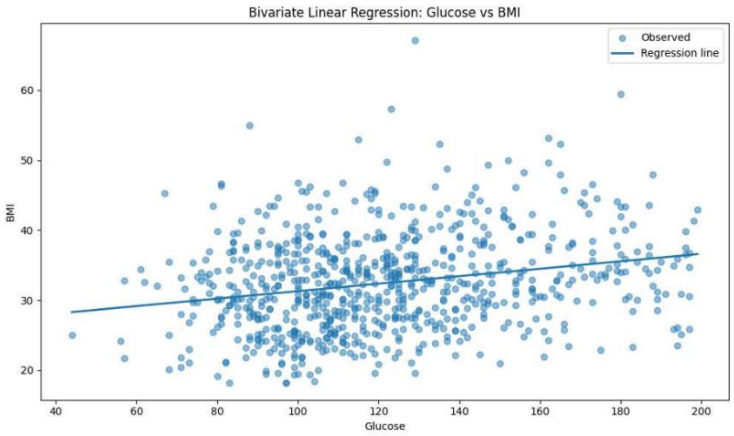
> *Note for viva: the two bivariate models were scored on the same data they were trained on, whereas the last model was scored on unseen test data. That is why the last accuracy (0.70) is lower and more honest.*

3. What the diagrams represent

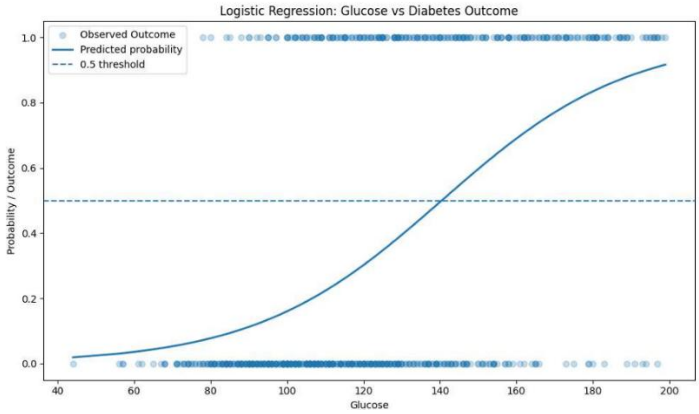
1. **Four histograms (Glucose, BMI, Age, BloodPressure):** show the distribution shape. Glucose, BMI and BloodPressure are roughly bell-shaped; Age is clearly right-skewed (many young patients, a few old).



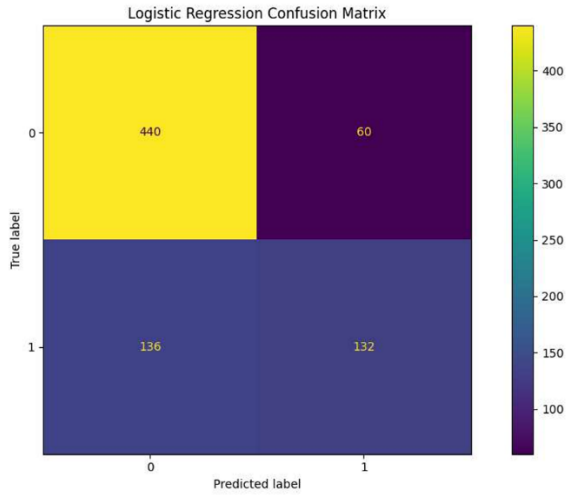
2. **Glucose vs BMI scatter with regression line:** a cloud of points with a nearly flat line. The wide scatter around the line visualises the low R^2 of 0.056.



3. **Logistic curve plot:** points at 0 and 1 on the y-axis, an S-shaped curve giving P(diabetes) rising with Glucose, and a dashed line at 0.5. Where the curve crosses 0.5 (Glucose of about 140) the prediction flips from 0 to 1.



4. **Confusion matrices (two):** a 2x2 grid of true vs predicted class. Diagonal cells are correct predictions; the off-diagonal cells are errors (false positives and false negatives). The large false-negative count explains the low recall.



4. Top 5 viva questions

Q1. Difference between linear and logistic regression? Linear regression predicts a continuous value (BMI) using a straight line and minimises squared error. Logistic regression predicts the probability of a class (diabetic or not) using the sigmoid function p = 1 / (1 + e^-(b0 + b1x)) and is trained by maximum likelihood; the probability is converted to a class using a threshold (0.5).

Q2. What do R^2, MAE and RMSE mean? Is $R^2 = 0.056$ good? R^2 is the fraction of variance in the response explained by the model (0 to 1). MAE is the average absolute error; RMSE is the square root of the average squared error and penalises large errors more. $R^2 = 0.056$ is very poor: Glucose alone explains only about 5.6% of BMI, so we added more variables, which raised R^2 to 0.389.

Q3. Explain accuracy, precision, recall and F1. Why is recall low here? Accuracy = correct / total. Precision = TP / (TP + FP). Recall = TP / (TP + FN). F1 is their harmonic mean. Recall is only about 0.49 to 0.52 because the data is imbalanced (500 vs 268) and the model, with a 0.5 threshold, misses many diabetic patients. In medicine recall matters most, so the threshold could be lowered or class weights used.

Q4. What are skewness and kurtosis, and what do mean, median and mode tell you? Skewness measures asymmetry (positive means a long right tail, e.g. Age); kurtosis measures tail heaviness/peakedness. If mean > median > mode the data is right-skewed, if they are almost equal the data is nearly symmetric.

Q5. Why use train_test_split with stratify and random_state? To evaluate on unseen data and detect overfitting. `stratify` keeps the same class proportion in train and test, which is important with imbalanced classes; `random_state` makes the split reproducible.

Practical 3: Handwritten Digit Recognition using SVM

1. Aim and idea

Classify images of handwritten digits (0 to 9) with a **Support Vector Machine (SVM)**. The `load_digits` dataset has 1797 images, each 8 x 8 pixels, flattened to 64 features with pixel values 0 to 16. An SVM finds the hyperplane that separates classes with the **maximum margin**; the training points lying on the margin are the **support vectors**. Kernels let it separate data that is not linearly separable.

2. Code line by line

Cell 1: imports

- `time` : to measure training time.
- `seaborn as sns` : nicer statistical plots (heatmap).
- `load_digits` : built-in digits dataset.
- `train_test_split, cross_val_score, GridSearchCV, StratifiedKFold` : splitting, cross-validation, hyper-parameter search, class-balanced folds.
- `StandardScaler`, `Pipeline` : scaling and chaining steps.
- `SVC` : Support Vector Classifier. `PCA` is imported but not used.

- Metrics: accuracy, precision, recall, F1, classification report, confusion matrix.

Cell 2: load data

- `digits = load_digits()` loads the dataset object.
- `X, y = digits.data, digits.target` : features (1797 x 64) and labels (1797).
- Prints shapes, the image array shape (1797, 8, 8), classes 0 to 9 and pixel range 0.0 to 16.0.

Cell 3: sample images

- `fig, axes = plt.subplots(2, 10, figsize=(15, 3.5))` : a grid of 2 rows x 10 columns.
- `for digit, ax_col in zip(range(10), axes.T)` : one column of the grid per digit.
- `idx = np.where(y == digit)[0][:2]` : indices of the first two images of that digit.
- `ax.imshow(digits.images[i], cmap="gray_r")` : shows the 8x8 image in reversed grey (dark ink on white).
- `ax.set_title(...)`, `ax.axis("off")` : label and hide axes.

Cell 4: train/test split

- `RANDOM_STATE = 42` fixes randomness.
- `train_test_split(X, y, test_size=0.20, stratify=y, random_state=...)` : 1437 training and 360 test images, with every digit class in the same proportion in both sets.
- `np.bincount(y_train) / len(y_train) * 100` : percentage of each class (all about 10%), proving the stratified split worked.

Cell 5: baseline SVM

- `Pipeline([("scaler", StandardScaler()), ("svm", SVC(kernel="rbf", C=1.0, gamma="scale"))])` : first standardise, then SVM with the RBF kernel. Putting the scaler inside the pipeline means it is fitted only on training data in every fold (no data leakage).
- `baseline.fit(X_train, y_train)` between two `time.time()` calls measures training time (0.074 s).
- Test accuracy **0.9750**. `n_support_.sum()` shows **746 of 1437** training points became support vectors.

Cell 6: compare kernels

- A dictionary of four SVMs: `linear`, `poly (degree 3)`, `rbf`, `sigmoid`.
- `StratifiedKFold(n_splits=5, shuffle=True, ...)` : 5-fold cross-validation keeping class balance.
- Loop: wrap each model in a Pipeline, `cross_val_score(pipe, X_train, y_train, cv=skf)` gives 5 accuracies, then fit on all training data, predict on test and store CV mean, CV std, test accuracy, macro F1 and time.
- Result: RBF CV 0.9812, linear 0.9770, poly 0.9485, sigmoid 0.9450 (test accuracy 0.9750, 0.9750, 0.9722, 0.9389).

Cell 7: bar chart

- `kernel_df[["CV mean acc","Test acc"]].plot(kind="bar")` draws grouped bars; `set_ylim(0.8, 1.01)` zooms into the useful range; the `for p in ax.patches` loop writes the value on top of each bar.

Cell 8: hyper-parameter tuning with GridSearchCV

- `param_grid` has three groups: RBF with C in {0.1, 1, 10, 100} and gamma in {0.0001, 0.001, 0.01, 0.1} (16 combinations); poly with C in {0.1, 1, 10} and degree in {2, 3, 4} (9); linear with C in {0.01, 0.1, 1, 10} (4). Total **29 candidates**. The prefix `svm__` tells the pipeline which step the parameter belongs to.
- `GridSearchCV(pipe, param_grid, cv=skf, scoring="accuracy", n_jobs=-1)` tries all candidates with 5-fold CV using all CPU cores.
- `grid.fit(...)` runs the search (28.5 s). Best: **poly kernel, C = 10, degree 3**, CV accuracy 0.9826.

Cell 9: final evaluation

- `best_model = grid.best_estimator_` : the best pipeline, already refitted on all training data.
- Test accuracy **0.9944**, macro precision 0.9945, recall 0.9944, F1 0.9944. `classification_report` shows per-class scores: nearly all 1.000, only digits 6 and 7 slightly lower, i.e. only 2 of 360 test images were wrong.

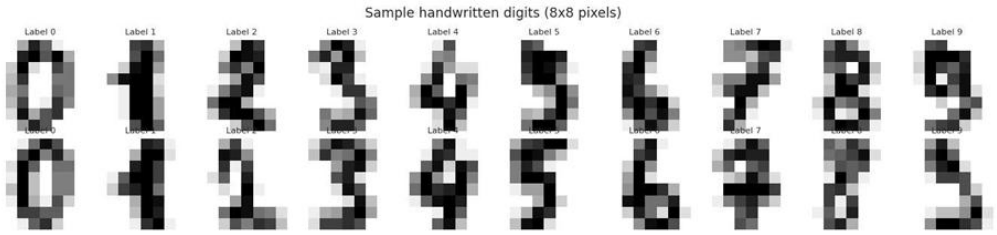
Cell 10: confusion matrices

- `confusion_matrix(y_test, y_pred)` then `ConfusionMatrixDisplay(...).plot(...)` for counts, and `cm / cm.sum(axis=1, keepdims=True)` divides each row by its total to get per-class recall, drawn with `sns.heatmap(annot=True, fmt=".2f")`.

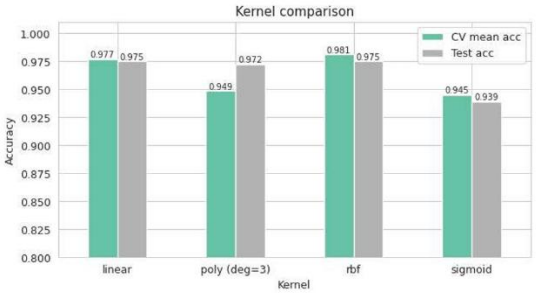
Cell 11: sample predictions

- `rng.choice(len(y_test), 20, replace=False)` picks 20 random test images; each is reshaped `X_test[i].reshape(8, 8)` and shown with its predicted label, **green if correct, red if wrong**.

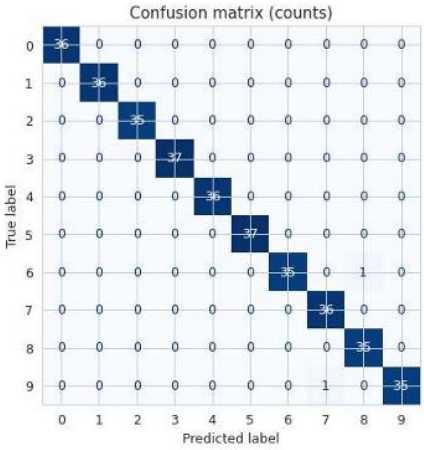
3. What the diagrams represent

1. **Sample digits grid (2 x 10):** two example 8x8 images per digit. They are very low resolution, which shows why the task is non-trivial.
2. 

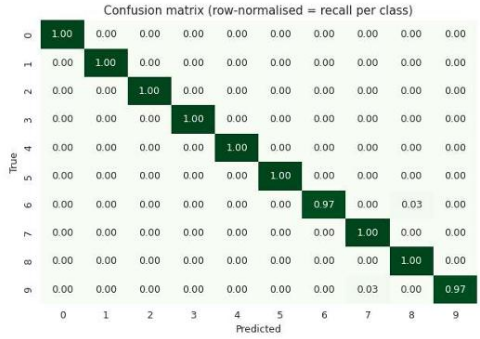
3. **Kernel comparison bar chart:** for each kernel, one bar for cross-validated accuracy and one for test accuracy. RBF and linear are best with default settings; sigmoid is lowest.



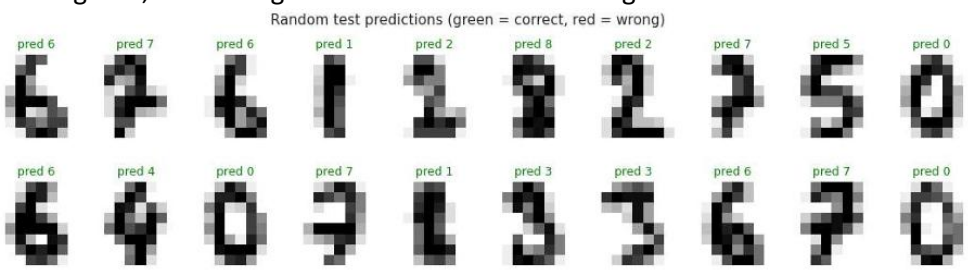
4. **Confusion matrix (counts, blue):** rows are true digits and columns predicted digits. A strong diagonal means almost every image is classified correctly; the one or two off-diagonal entries are the mistakes.



5. **Row-normalised heatmap (green):** the same matrix expressed as fractions, so the diagonal values are the recall of each digit (1.00 for most, 0.97 for the weaker digit).



6. **Random test predictions grid:** 20 digit images with the predicted label above each; nearly all titles are green, confirming the model reads handwriting well.



4. Top 5 viva questions

Q1. What is SVM? What are the margin and support vectors? SVM is a supervised algorithm that finds the hyperplane separating classes with the largest possible margin (distance to the nearest points of each class). Those nearest points are the support vectors; only they define the boundary. In your run 746 of 1437 points were support vectors.

Q2. What is the kernel trick? Name the kernels you tried. A kernel computes the dot product of points in a higher-dimensional space without actually transforming the data, so non-linear boundaries can be learned. I compared linear, polynomial, RBF (Gaussian) and sigmoid kernels. RBF gave the best cross-validation accuracy by default, and polynomial with C = 10 was best after tuning.

Q3. What do the parameters C and gamma control? C is the penalty for misclassification: small C gives a wide, soft margin (more tolerance, risk of underfitting); large C gives a narrow margin that fits training data tightly (risk of overfitting). Gamma, in RBF/poly/sigmoid kernels, sets how far the influence of one training point reaches: high gamma means very local, wiggly boundaries (overfitting); low gamma gives smoother ones.

Q4. Why is feature scaling needed, and why did you use a Pipeline? SVM is distance-based, so features on larger scales would dominate. A Pipeline applies the scaler and model together and, inside cross-validation, fits the scaler only on the training folds, which avoids data leakage.

Q5. What is GridSearchCV and stratified k-fold cross-validation? K-fold CV splits the training data into k parts, trains on k-1 and validates on the remaining one, rotating k times and averaging; the stratified version keeps class proportions in each fold. GridSearchCV tries every combination in a parameter grid using this CV and returns the best (here 29 combinations x 5 folds).

Practical 4: K-Means Clustering on Iris (Elbow Method)

1. Aim and idea

Group the 150 iris flowers into clusters **without using the species label** (unsupervised learning). **K-Means** picks k centroids, assigns each point to its nearest centroid, moves each centroid to the mean of its points, and repeats until nothing changes. The **Elbow method** chooses k by plotting WCSS (within-cluster sum of squares, called inertia) against k and looking for the bend.

2. Code line by line

Cell 1: imports

- `os` : file checks. `KMeans` : the algorithm. `StandardScaler` : scaling.
- `silhouette_score, calinski_harabasz_score, davies_bouldin_score` : internal cluster-quality metrics (no labels needed).
- `adjusted_rand_score, normalized_mutual_info_score, homogeneity_score, ...` : external metrics comparing clusters with true species.

- `linear_sum_assignment` : Hungarian algorithm to match cluster numbers to species. `warnings.filterwarnings("ignore")` hides warnings; `sns.set_style("whitegrid")` sets plot style.

Cell 2: load data

- `if os.path.exists("Iris.csv")` reads the file if present; otherwise (the `else` branch) `load_iris(as_frame=True)` loads the same data from scikit-learn, renames the columns to SepalLengthCm etc., maps the numeric target to species names with `.map(dict(enumerate([...])))`, drops `target` and inserts an `Id` column. *Your output says "Iris.csv not found", so the sklearn copy was used.*

Cell 3: checks

- Prints shape (150, 6), dtypes, missing values (none), duplicate rows ignoring Id (1) and species counts (50 each).

Cell 4: `df.describe().round(2)` gives the summary statistics.

Cell 5: heatmap and box plot

- `features = [...]` : the 4 measurement columns.
- `sns.heatmap(df[features].corr(), annot=True, cmap="coolwarm", center=0)` : correlation matrix coloured from blue (negative) to red (positive).
- `df[features].plot(kind="box")` : box plots to spot outliers and compare spreads.

Cell 6: `sns.pairplot(df.drop(columns="Id"), hue="Species", diag_kind="kde")` : every feature against every other, coloured by true species. It is only for understanding; K-Means never sees the species.

Cell 7: scaling

- `X = df[features].values` : feature matrix as NumPy array.
- `y_true = pd.factorize(df["Species"])[0]` : species converted to 0, 1, 2, kept **only for evaluation**.
- `X_scaled = StandardScaler().fit_transform(X)` : mean 0, std 1 (printed to confirm).

Cell 8: first model

- `KMeans(n_clusters=3, init="k-means++", n_init=10, random_state=42).fit(X_scaled)`: `init="k-means++"` spreads the starting centroids far apart; `n_init=10` runs 10 different starts and keeps the best. `km_sk.n_iter_` = 4 iterations; `km_sk.inertia_` = WCSS = 139.820.

Cell 9: elbow loop

- `for k in range(1, 11)`: fit K-Means for k = 1 to 10 and append `km.inertia_` to the list `wcss`.
- The table adds `.diff().abs()` (drop from the previous k) and the % drop. WCSS: k=1: 600.00, k=2: 222.36, k=3: 139.82, k=4: 114.09 ... k=10: 47.39. The drop is big up to k=3 and small afterwards.

Cell 10: automatic knee detection

- `pts = np.c_[list(K_RANGE), wcss]` : (k, WCSS) points. `p1, p2` : first and last point.
- `line = (p2 - p1) / np.linalg.norm(p2 - p1)` : unit vector along the line joining them.
- `dist_to_line = np.abs(vec[:,0]*line[1] - vec[:,1]*line[0])` : perpendicular distance of every point from that line.
- `knee_k = int(pts[dist_to_line.argmax(), 0])` : the k farthest from the line is the elbow, **k = 3**. The plot draws the WCSS curve, the grey dashed line and a red marker at k = 3.

Cell 11: final model

- `KMeans(n_clusters=3, init="k-means++", n_init=20, max_iter=300, ...)`; `df["Cluster"] = kmeans.fit_predict(X_scaled)` trains and stores each flower's cluster.
- Outputs: 4 iterations, WCSS 139.820, **silhouette score 0.460**, sizes 53 / 50 / 47.
- `scaler.inverse_transform(kmeans.cluster_centers_)` converts the centroids back to centimetres so they can be read: Cluster 1 (5.01, 3.43, 1.46, 0.25) is small-petal = setosa-like; Cluster 2 (6.78, 3.10, 5.51, 1.97) is the largest flowers = virginica-like; Cluster 0 is in between = versicolor-like.

Cell 12: `df.groupby("Cluster")[features].mean()` : average of every feature per cluster (the cluster profile).

Cell 13: cluster scatter plots

- Two panels: Sepal length vs width and Petal length vs width. Each cluster gets its own colour (`palette`), and the centroids are gold X markers (`marker="X", s=350`).

Cell 14: cluster vs species

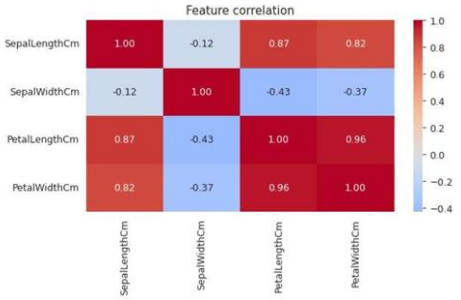
- `ct = pd.crosstab(df['Species'], df['Cluster'])` : count table of species against cluster; `sns.heatmap(ct, annot=True, fmt="d")` draws it.
- **Warning:** the next lines use a variable `mapped` that is never defined, so your notebook shows `NameError: name 'mapped' is not defined`. Fix it by adding these lines before `wrong = ...`:

```
cm = confusion_matrix(y_true, df["Cluster"])
row_ind, col_ind = linear_sum_assignment(-cm)
mapping = dict(zip(col_ind, row_ind))
mapped = df["Cluster"].map(mapping).values
```

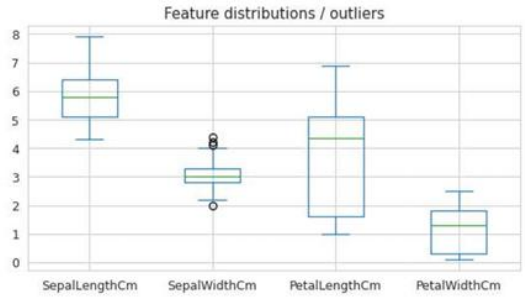
The Hungarian algorithm matches each cluster number to the species it overlaps most; `wrong = np.where(mapped != y_true)[0]` then lists the flowers sitting in a "wrong" cluster (versicolor/virginica border cases).

3. What the diagrams represent

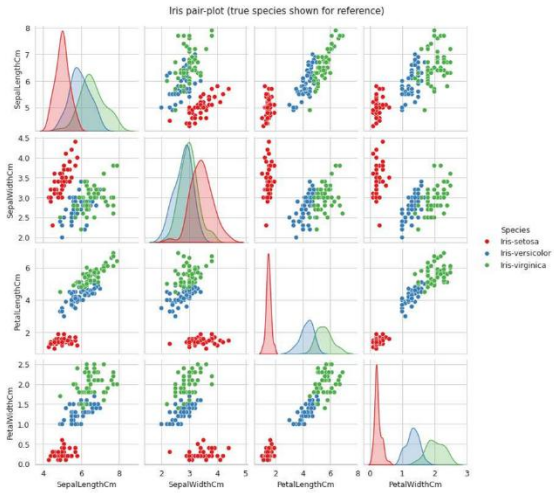
1. **Correlation heatmap:** numbers from -1 to 1. Petal length and petal width are very highly correlated (about 0.96), and both correlate strongly with sepal length; sepal width is weakly or negatively correlated.



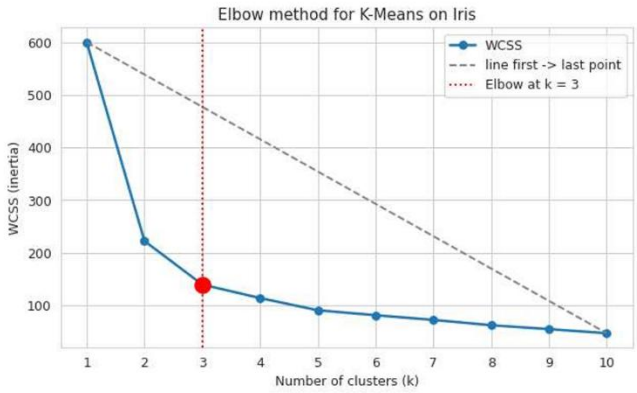
2. **Box plot:** the spread of each feature; the box is the middle 50%, the line is the median, dots outside are outliers (sepal width has a few).



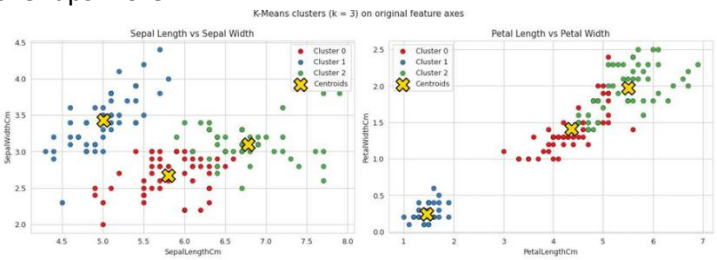
3. **Pair plot:** a grid of scatter plots with density curves on the diagonal. Setosa (red) is clearly separated from the other two, while versicolor (blue) and virginica (green) overlap. This predicts that K-Means will separate setosa perfectly and mix a few of the other two.



4. **Elbow plot:** WCSS falls steeply from k=1 to k=3, then flattens; the red marker shows the elbow at **k = 3**, the best trade-off between few clusters and low error.



5. **Cluster scatter plots with centroids:** the 3 coloured groups in original units. Gold X marks are the centroids (the mean of each cluster). The petal plot shows three neat bands; the sepal plot overlaps more.



6. **Species vs cluster heatmap:** a table of counts. One row/column is clean (setosa to one cluster), while versicolor and virginica are split across two clusters, showing the few mismatches.



4. Top 5 viva questions

Q1. Explain how the K-Means algorithm works. (1) Choose k and initial centroids (k-means++). (2) Assign every point to the nearest centroid (Euclidean distance). (3) Recompute each centroid as the mean of its assigned points. (4) Repeat 2 and 3 until assignments stop changing or `max_iter` is reached. It minimises WCSS.

Q2. What is the elbow method and what is WCSS? WCSS (inertia) is the sum of squared distances between each point and its cluster centroid. It always decreases as k increases, so we plot it against k

and choose the "elbow" where the improvement suddenly becomes small. Here WCSS dropped from 600 to 222 to 140 and then slowly, so k = 3.

Q3. Why did you scale the data, and what does k-means++ do? K-Means uses distance, so features with larger ranges dominate; scaling gives all features equal weight. k-means++ chooses initial centroids far apart, which gives faster convergence and avoids poor local minima compared with purely random starts.

Q4. What is the silhouette score? What does 0.46 mean? For each point, silhouette = (b - a) / max(a, b), where a is the mean distance to its own cluster and b the mean distance to the nearest other cluster. It lies between -1 and 1: near 1 means well separated, near 0 means overlapping clusters. 0.46 is moderate: reasonable clustering, with overlap between versicolor and virginica.

Q5. What are the limitations of K-Means? Supervised vs unsupervised? K must be chosen in advance, it assumes roughly spherical clusters of similar size, is sensitive to outliers and initialisation, and only finds local minima. It is unsupervised: no labels are used for training (species were used only afterwards to check the result), unlike supervised models such as SVM or logistic regression.

Practical 5: Ensemble Learning (AdaBoost, Gradient Boosting, XGBoost and Voting) on Iris

1. Aim and idea

Ensemble learning combines several models to get better and more stable predictions than a single model. Here three boosting models (AdaBoost, Gradient Boosting, XGBoost) are trained on Iris, and then combined with **hard voting** and **soft voting**. Boosting builds weak learners one after another, each focusing on the mistakes of the previous ones.

2. Code line by line

Cell 1: imports

- `os`, `warnings` (`filterwarnings("ignore")` hides warnings), `numpy`, `pandas`, `matplotlib`, `seaborn`.
- `load_iris` : data; `train_test_split` : split; `LabelEncoder` : text labels to numbers.
- Metrics : accuracy, precision, recall, F1, classification report, confusion matrix.
- `AdaBoostClassifier, GradientBoostingClassifier, VotingClassifier` : sklearn ensembles; `DecisionTreeClassifier` : weak learner; `XGBClassifier` : XGBoost library model.
- `RANDOM_STATE = 42` and `np.random.seed(42)` : reproducibility.

Cell 2: load data

- `csv_path = ""` is empty, so the `else` branch runs: `load_iris(as_frame=True)` then `iris_df = iris.frame.copy()` and the numeric target is mapped to names (setosa, versicolor, virginica) with `.map(dict(enumerate(iris.target_names)))`.
- Prints the head, shape (150, 5), columns, missing values (0) and class counts (50 each).

Cell 3: features, labels and split

- `target_col = "Species" if ... else "target"` : finds the label column name.
- `X = iris_df.drop(columns=[target_col]); y_raw = ...astype(str)`.
- `X.select_dtypes(include=np.number)` : keeps only numeric columns (guards against ID columns).
- `label_encoder.fit_transform(y_raw)` : setosa=0, versicolor=1, virginica=2.
- `train_test_split(..., test_size=0.20, random_state=42, stratify=y)` : **120 training and 30 test samples** (10 of each class).

Cell 4: `sns.pairplot(plot_df, hue="Species", corner=True)` draws only the lower triangle of the feature-vs-feature plots coloured by species.

Cell 5: define and train the three models

- **AdaBoost:** `AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1), n_estimators=100, learning_rate=0.5)`: 100 decision *stumps* (trees of depth 1); each new stump gives more weight to the samples misclassified so far; `learning_rate` shrinks each stump's contribution.
- **Gradient Boosting:** `GradientBoostingClassifier(n_estimators=100, learning_rate=0.05, max_depth=2)`: 100 small trees, each fitted to the errors (gradient of the loss) of the current ensemble; small learning rate = slow, careful learning.
- **XGBoost:** `XGBClassifier(n_estimators=100, max_depth=3, learning_rate=0.05, subsample=0.9, colsample_bytree=0.9, objective="multi:softprob", eval_metric="mlogloss", num_class=3, n_jobs=2)`: an optimised gradient boosting with regularisation. `subsample=0.9` uses 90% of rows per tree, `colsample_bytree=0.9` uses 90% of columns per tree; `multi:softprob` outputs class probabilities for multi-class; `mlogloss` is the evaluation loss.
- The `models` dictionary and the `for` loop call `model.fit(X_train, y_train)` on each.

Cell 6: evaluation function

- `evaluate_model` predicts on the test set and returns accuracy and macro-averaged precision, recall and F1 (`average="macro"` = simple average over the three classes; `zero_division=0` avoids errors if a class is never predicted).
- Results: **AdaBoost 0.9667, Gradient Boosting 0.9333, XGBoost 0.9667** accuracy.

Cell 7: `classification_report(y_test, predictions[name], target_names=...)` prints per-class precision, recall, F1. Setosa is perfect in all models; the errors are one versicolor/virginica mix-up in AdaBoost and XGBoost and two in GBM.

Cell 8: confusion matrices of the 3 models

- `sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, xticklabels=..., yticklabels=...)` in a 1 x 3 grid.

Cell 9: voting ensembles

- `voting_estimators` : a list of (name, model) pairs for the three boosting models.

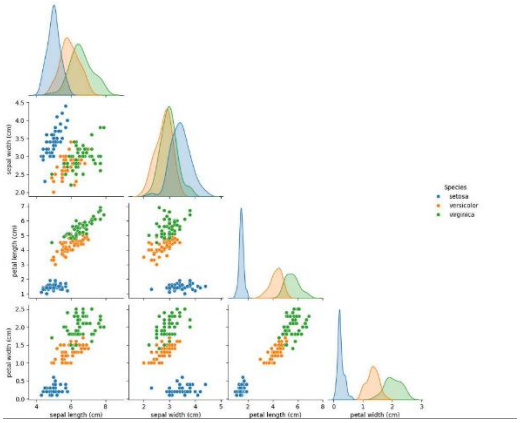
- `VotingClassifier(estimators=voting_estimators, voting="hard")` : each model casts one vote for a class label and the **majority wins**.
- `VotingClassifier(..., voting="soft")` : the models' predicted **probabilities are averaged** and the class with the highest average wins (all three models have `predict_proba`).
- Both are fitted and evaluated: **Hard voting 0.9667, Soft voting 0.9333.**

Cell 10: `pd.concat([...])` joins single-model and voting results into one table of 5 models, then `.plot(kind="bar")` draws a grouped bar chart of the four metrics.

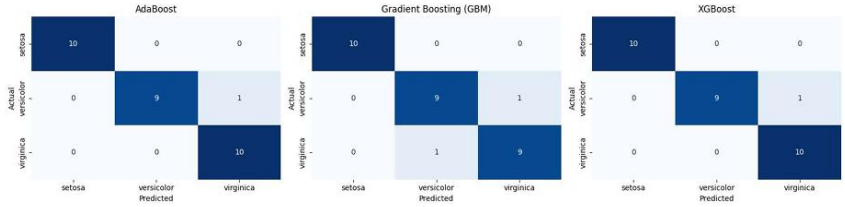
Cell 11: confusion matrices (green) for Hard and Soft voting.

3. What the diagrams represent

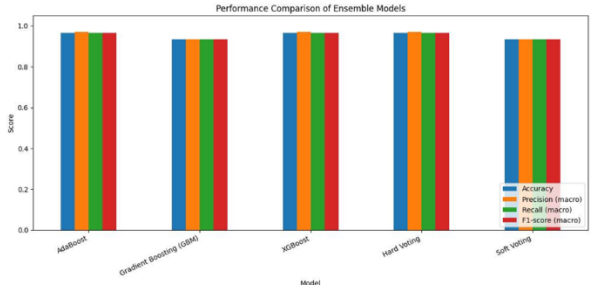
1. **Pair plot (corner):** scatter plots of every pair of the 4 features, coloured by species, and distributions on the diagonal. Setosa is far from the rest; versicolor and virginica touch each other. That is where all the classification errors occur.



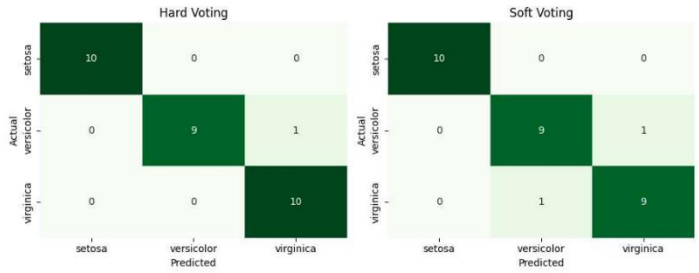
2. **Confusion matrices of AdaBoost, GBM, XGBoost:** 3 x 3 tables. The diagonal shows correct predictions (10 for setosa in all). AdaBoost and XGBoost have a single error (a versicolor predicted as virginica), GBM has two errors.



3. **Bar chart of model performance:** grouped bars (accuracy, precision, recall, F1) for the five models. All bars are above 0.93; AdaBoost, XGBoost and Hard Voting tie at the top (0.9667), while GBM and Soft Voting are at 0.9333.



4. **Voting confusion matrices:** the same type of 3 x 3 table for Hard and Soft voting, showing the exact samples confused.



> *Note: the test set has only 30 samples, so one sample equals 3.3% accuracy. The differences between the models are therefore not statistically meaningful; say this if the examiner asks "which model is best?".*

4. Top 5 viva questions

Q1. What is ensemble learning? Difference between bagging and boosting? Ensemble learning combines multiple models to improve accuracy and reduce overfitting. Bagging (e.g. Random Forest) trains models independently on bootstrap samples in parallel and averages them to reduce variance. Boosting trains models sequentially, each correcting the errors of the previous one, to reduce bias.

Q2. How does AdaBoost work? It starts with equal sample weights, trains a weak learner (a decision stump), increases the weights of misclassified samples, trains the next learner on the re-weighted data, and finally combines all learners by a weighted vote where more accurate learners get a larger say.

Q3. How does Gradient Boosting differ from AdaBoost, and what is XGBoost? AdaBoost re-weights samples; Gradient Boosting fits each new tree to the residual errors (negative gradient of a loss function) of the current model. XGBoost is an optimised, regularised (L1/L2) implementation of gradient boosting with second-order gradients, parallel tree building, built-in handling of missing values, and row/column subsampling.

Q4. What is the difference between hard and soft voting? Which is better? Hard voting takes the majority of predicted class labels; soft voting averages predicted probabilities and picks the highest. Soft voting usually performs better because it uses model confidence, but it needs well-calibrated probabilities. In your run hard voting scored 0.9667 and soft 0.9333, a difference of just one test sample.

Q5. What do learning_rate and n_estimators do? n_estimators is the number of boosting rounds (trees). learning_rate scales the contribution of each tree. They trade off: a small learning rate (0.05) needs more trees but generalises better; a large one learns quickly but may overfit.

Practical 6: Reinforcement Learning, Tic-Tac-Toe with Q-Learning

1. Aim and idea

In **reinforcement learning** an **agent** interacts with an **environment**, takes **actions** in **states**, and receives **rewards**; it learns a policy that maximises total reward. Here two Q-learning agents (X and O) learn Tic-Tac-Toe by **self-play**. The **Q-table** stores a value Q(state, action) for every board position and every move; learning uses the update

Q(s,a) = Q(s,a) + alpha x [r + gamma x max Q(s',a') - Q(s,a)]

where alpha is the learning rate, gamma the discount factor, r the reward, and s' the next state.

2. Code line by line

Cell 1: imports

- `numpy`, `pandas`, `random` (random choices), `defaultdict` (a dictionary that creates a default value for missing keys), `matplotlib`. `random.seed(42)` and `np.random.seed(42)` make the run repeatable.

Cell 2: the environment class `TicTacToe`

- `__init__` calls `self.reset()`.
- `reset()` : `self.board = np.zeros(9, dtype=np.int8)` creates an empty board as 9 cells (0 = empty, 1 = X, -1 = O); `self.current_player = 1` means X starts; returns the state.
- `get_state()` : `tuple(self.board.tolist())` converts the board to a tuple, which can be used as a dictionary key for the Q-table.
- `available_actions()` : list of indices of empty cells (`value == 0`).
- `check_winner()` : `winning_lines` lists the 8 winning triples (3 rows, 3 columns, 2 diagonals). For each (a, b, c), if the cell is non-zero and all three cells are equal, return that player (1 or -1). If no empty cells remain it returns `0` (draw), otherwise `None` (game continues).
- `step(action)` : raises an error for an occupied cell; otherwise places the current player's mark, checks the result, switches player with `self.current_player *= -1` if the game goes on, and returns `(new_state, result, done)`.
- `render()` : prints the board with symbols X, O and blank, separating rows with lines.
- Last lines create `env`, print the initial state `(0,0,0,0,0,0,0,0,0)`, the 9 available actions and an empty board.

Cell 3: rule test. Moves `[0, 3, 1, 4, 2]` are played alternately (X: 0, 1, 2; O: 3, 4), so X wins the top row. Output confirms **Winner: X, Game over: True**, proving the game logic works.

Cell 4: the `QLearningAgent` class

- `__init__` parameters: `alpha=0.2` (learning rate), `gamma=0.95` (discount factor), `epsilon=1.0` (start with 100% exploration), `epsilon_min=0.05`, `epsilon_decay=0.9995`. `self.q_table = defaultdict(lambda: np.zeros(9))` creates 9 zero Q-values for any new state automatically.
- `choose_action(state, available_actions, training=True)` implements **epsilon-greedy**: with probability epsilon (`random.random() < self.epsilon`) pick a random legal move

(**explore**); otherwise take the legal move with the highest Q-value (**exploit**). `best_actions` collects all actions tied for the maximum and chooses one randomly (tie-breaking).

- `update(state, action, reward, next_state, next_actions, done)` : `current_q` is the old value; if the game is over `target = reward`, else `target = reward + gamma * max Q(next_state, a)` over legal next actions; then `Q += alpha * (target - current_q)`. This is the Q-learning rule above.
- `decay_exploration()` : `epsilon = max(epsilon_min, epsilon * epsilon_decay)` reduces exploration after every game but never below 0.05.
- Two agents are created: `agent_x` and `agent_o`.

Cell 5: self-play training `train_self_play(agent_x, agent_o, episodes=20000)`

- `history` dictionary and block counters record wins/draws for every 100 games.
- For each episode: a fresh `env`; `last_transition = {1: None, -1: None}` remembers each player's previous (state, action).
- Inside `while True`: pick the player and agent, get legal moves, choose an action, call `env.step(action)`.
- **Delayed update for the opponent:** a player's move only gets its true next state after the *other* player has moved. So after each move, the opponent's stored previous transition is updated: if the game ended with a win for the current player, the opponent receives reward **-1** (loss), a draw gives 0; otherwise reward 0 with the new state.
- If the game is `done`, the current player's last move gets reward **+1** for a win, 0 for a draw; the winner/draw counter is incremented and the loop breaks.
- Otherwise the current transition is stored and `state` moves on.
- After each episode both agents call `decay_exploration()`; every 100 episodes the counts and epsilon are saved to `history` and the counters reset.
- Results: **X learned 2181 states, O learned 1907 states**, final epsilon 0.05.

Cell 6: training plots. `np.arange(100, 20001, 100)` is the x-axis (every 100 episodes). The first figure plots X wins, O wins and draws per 100 games; the second plots epsilon.

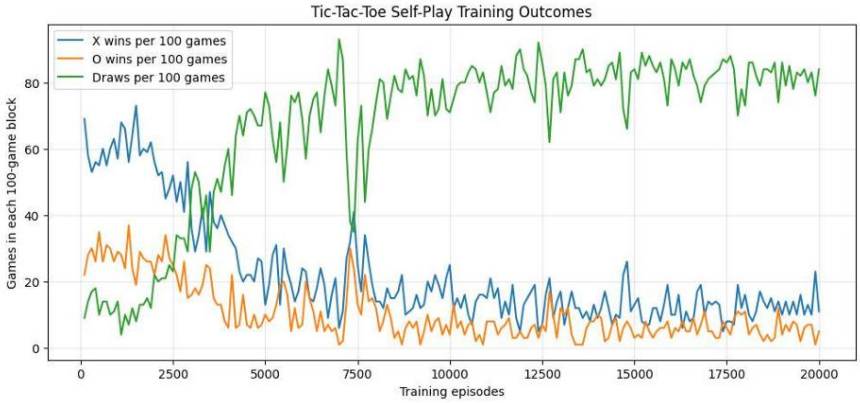
Cell 7: testing `play_against_random(agent, games=1000)`

- Agent X plays with `training=False` (pure exploitation, no random moves) against a random O. Counts wins, losses, draws.
- Result: **844 wins, 51 losses, 105 draws**: win rate 0.844, draw rate 0.105, non-loss rate **0.949**.

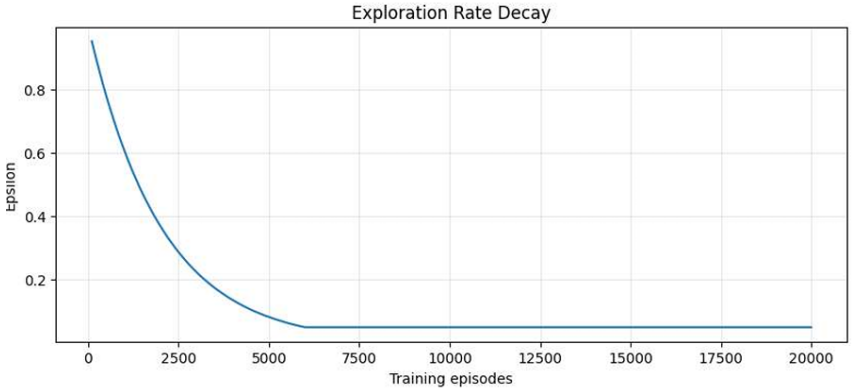
Cell 8: demo game. Prints each board after every move: the agent X takes positions 4, 5 and 6 (the middle row) and wins against a random opponent.

3. What the diagrams represent

1. **Self-play outcomes per 100 games (3 lines):** at the start X wins about 55 to 70 of every 100 games, because both agents play almost randomly and X moves first. As training proceeds the **draw line climbs to about 80 to 90 per 100** while X and O wins fall to roughly 5 to 20. This is the expected result: with good play on both sides Tic-Tac-Toe is a draw. The dip in draws around episode 7000 is temporary instability while the agents adjust to each other.



2. **Exploration rate (epsilon) decay:** a smooth curve falling from 1.0 and flattening at 0.05 after about 6000 episodes (0.9995 raised to the number of episodes). It shows the shift from exploring (random moves) to exploiting (best-known moves).



3. **Rendered board / demo game:** text boards showing the sequence of moves in one game, proving the trained agent plays sensibly.
4. **Test table:** 844 wins of 1000 against a random opponent, showing the learned Q-table is useful.

4. Top 5 viva questions

Q1. What is reinforcement learning? Name its main components. A type of machine learning where an agent learns by trial and error from rewards. Components: agent (the learner), environment (the game), state (board position), action (a move), reward (+1 win, -1 loss, 0 otherwise) and policy (the rule for choosing actions).

Q2. What is Q-learning and what is the update rule? Q-learning is a model-free, off-policy, value-based algorithm that learns Q(s,a), the expected future reward of taking action a in state s. Update: Q(s,a) = Q(s,a) + alpha x [r + gamma x max Q(s',a') - Q(s,a)]. In the code: `self.q_table[state][action] += self.alpha * (target - current_q)`.

Q3. What are alpha, gamma and epsilon? Alpha (0.2) is the learning rate: how strongly new information overrides the old Q-value. Gamma (0.95) is the discount factor: how much future rewards matter versus immediate ones. Epsilon is the exploration probability in epsilon-greedy action selection; it starts at 1.0 and decays by 0.9995 per game to a minimum of 0.05.

Q4. What is the exploration vs exploitation trade-off? Exploration means trying random actions to discover better strategies; exploitation means using the best action known so far. Too little exploration can get stuck in a poor policy, too much never uses what was learned. Epsilon-greedy with decay explores a lot early and exploits more later.

Q5. How did the agents learn, and how did you evaluate them? Two agents played 20,000 games against each other (self-play), updating their Q-tables after each move with +1 / -1 / 0 rewards. Evaluation: the trained X agent (no exploration) played 1000 games against a random player: 844 wins, 105 draws and 51 losses, a non-loss rate of 94.9%. A tabular method works here because Tic-Tac-Toe has a small state space; larger games would need function approximation (Deep Q-Networks).
